Agentic AI Research & Task Execution System

Milestone-based implementation plan · 9–12 October 2026 · Submission deadline: Monday, 12 October 2026

4 days incl. today	**5 agents** Planner → Reviewer	**2 deployments** Render + Vercel	**Oct 12** submission

Objective: Deliver a working, deployed full-stack application that accepts a research task, plans subtasks, routes work through specialized agents, uses a real tool, stores workflow state, reviews the result, and displays a structured report with sources.

Delivery strategy: Prioritize a reliable MVP over extra features. Treat Sunday evening as the internal feature-complete deadline and reserve Monday for deployment, smoke tests, documentation and submission.

1. Recommended technology stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Task form, plan, execution timeline, agent outputs, report
Styling	Tailwind CSS + Lucide	Responsive UI and icons
Backend	Python + FastAPI + Pydantic	Typed REST APIs and validation (async where useful)
Orchestration	LangGraph	Agent routing, shared state, reviewer loop
LLM	Gemini, OpenAI or Claude API	Planning, analysis, writing, review
Search tool	Tavily or equivalent web-search API	Live research results and source URLs
Database	PostgreSQL	Tasks, execution events, agent outputs, history
Memory	LangGraph state (+ PostgreSQL)	Shared in-run context; persistent history
Hosting	Render (backend) + Vercel (frontend)	Public deployment

Scope note: FAISS/ChromaDB, document/PDF upload, RAG, report export and conversational memory are bonus features. Add them only after the required end-to-end workflow works.

2. System architecture

```
React + TypeScript frontend
   task input · execution timeline · agent outputs · final report
        │  HTTP API + status polling (every 2 s)
        ▼
FastAPI backend  —  POST /api/tasks · GET /api/tasks/{id} · GET /api/tasks/{id}/events
        ▼
LangGraph orchestrator (shared TaskState)
   1. Planner   →  goal → structured execution plan
   2. Research  →  web-search tool → findings + sources
   3. Analysis  →  insights, patterns, comparisons
   4. Writer    →  structured Markdown report
   5. Reviewer  →  pass ─► finalize
                   fail ─► Writer revises once ─► Reviewer again
        ▼
PostgreSQL  —  tasks · events · outputs · final report
```

Agent responsibilities

Agent	Responsibility	Input → Output
Planner	Understand the goal; create and delegate an actionable plan.	User request → typed subtasks with dependencies
Research	Call the web-search tool and collect relevant evidence.	Research subtasks → findings, URLs, snippets
Analysis	Compare findings; identify trends, gaps, risks and implications.	Findings → insights and conclusions
Writer	Produce a structured report or response.	Plan + findings + analysis → Markdown report
Reviewer	Check completeness, consistency, relevance and source support.	Draft + original request → pass/fail + feedback

Shared state (TaskState): task_id, user_request, plan, current_stage, research_findings, analysis, draft, review_feedback, revision_count, final_report, sources, errors, timestamps.

Reflection rule: if the Reviewer rejects the draft, its feedback goes back to the Writer for exactly one revision, then a second review. Cap revision_count at 1 to prevent infinite loops.

3. Four-day milestone schedule

FRIDAY · 9 OCT | 5–7 hours

Milestone 1 — Architecture & backend foundation

☐ Create GitHub repository with backend/ and frontend/ folders.

☐ Set up FastAPI, Pydantic, environment configuration and /health endpoint.

☐ Define TaskState and typed agent input/output schemas.

☐ Create PostgreSQL tables for tasks and execution events.

☐ Implement task creation, status and history API routes.

☐ Verify endpoints using Swagger/OpenAPI.

Exit criterion: backend runs locally, database is connected, API contract is documented.

SATURDAY · 10 OCT | 7–9 hours

Milestone 2 — Multi-agent orchestration

☐ Implement Planner Agent with structured task decomposition.

☐ Implement Research Agent with a working web-search tool.

☐ Implement Analysis Agent and Writer Agent (Markdown output).

☐ Implement Reviewer Agent with pass/fail decision and feedback.

☐ Wire LangGraph nodes, shared state and the single revision loop.

☐ Persist stage changes and agent outputs; run an end-to-end test.

Exit criterion: a real query flows through all five agents and produces a reviewed report.

SUNDAY · 11 OCT | 6–8 hours

Milestone 3 — Frontend & integration

☐ Build the responsive task-submission interface.

☐ Display generated plan, assigned agents, status and timeline (polling).

☐ Show individual agent outputs, tool usage and source links.

☐ Render final Markdown report and reviewer verdict.

☐ Add loading, empty, failure and retry states.

☐ Connect frontend to backend; test real tasks and browser refresh.

Exit criterion: complete local full-stack application with visible agent execution.

MONDAY · 12 OCT | 5–7 hours

Milestone 4 — Deployment, testing & submission

☐ Deploy FastAPI backend on Render and configure environment variables.

☐ Deploy React frontend on Vercel with the production API URL.

☐ Configure CORS, database connectivity and health checks.

☐ Run smoke tests against the deployed URLs.

☐ Write README: architecture, agent roles, flow, setup, env vars, API usage, deployment.

☐ Verify GitHub repository, prepare demo scenarios, submit all links.

Exit criterion: public frontend and backend URLs, complete repository and README, tested demo.

4. Suggested repository structure

```
agentic-ai-research/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/tasks.py
│   │   ├── agents/          planner, researcher, analyst, writer, reviewer
│   │   ├── orchestration/   graph.py, state.py
│   │   ├── tools/web_search.py
│   │   ├── models/schemas.py
│   │   ├── services/task_service.py
│   │   ├── db/database.py
│   │   └── core/config.py
│   ├── tests/
│   ├── requirements.txt
│   └── .env.example
├── frontend/
│   ├── src/  components/ pages/ hooks/ services/ types/ App.tsx
│   └── package.json
├── README.md
└── .gitignore
```

5. Backend API contract

Method	Endpoint	Purpose
POST	/api/tasks	Submit a new task and start execution
GET	/api/tasks/{id}	Task details and overall status
GET	/api/tasks/{id}/plan	Generated subtasks
GET	/api/tasks/{id}/events	Agent execution events (for UI polling)
GET	/api/tasks/{id}/result	Final reviewed report and sources
GET	/api/tasks	List previous tasks
GET	/health	Deployment health check

Status lifecycle: queued → planning → researching → analyzing → writing → reviewing → completed, plus failed for unrecoverable errors. Persist each transition so the UI can rebuild state after a refresh. Polling every 2 seconds is enough for v1; SSE/WebSockets are optional.

6. Frontend screens

View	Contents
Task submission	Natural-language input, example prompts, Start Research button
Execution monitor	Generated plan, agents involved, live status per stage, agent outputs, tool calls
Final report	Structured Markdown report, source links, reviewer verdict, revision note
History (bonus)	Previous tasks reloaded from PostgreSQL

7. Feature prioritization

Required MVP	Bonus — only after MVP works
Five specialized agents (minimum three)	RAG with document/PDF upload
Real tool integration (web search)	Vector DB (FAISS / ChromaDB)
Shared workflow state via LangGraph	Persistent conversational memory
Reviewer + one revision iteration	SSE / WebSocket live streaming
Persistent task history (PostgreSQL)	Report export (PDF/Markdown)
Full-stack UI: plan, status, outputs, report	Parallel research agents, metrics
Render + Vercel deployment and README	Redis job queue / worker

8. Testing and acceptance criteria

Test	Expected result
Normal research query	All five agents run; final report is structured and relevant.
Tool integration	Research Agent calls a real search API and stores working source URLs.
Reviewer requests revision	Writer revises once; the updated draft is reviewed again.
API / provider failure	Task shows a clear failed state; UI never spins indefinitely.
Browser refresh mid-run	Saved status and prior agent events remain visible.
Production configuration	Frontend uses deployed API URL; CORS works; no secrets in frontend or GitHub.

Deployment caution: don't rely on an in-process FastAPI background task as a durable job queue on Render. For the MVP, persist state before each stage and document the limitation; a dedicated worker + queue is the robust production design (a good talking point in the technical discussion).

9. Final submission checklist

☐ GitHub repository contains complete backend and frontend source.

☐ Backend deployed on Render; /health responds.

☐ Frontend deployed on Vercel and connected to the production API.

☐ At least three specialized agents working (target: five).

☐ A real tool is called and source links are visible.

☐ Agent state and outputs are passed between stages.

☐ Reviewer validates the result and can trigger one revision.

☐ Task plan, execution events and final result are visible in the UI.

☐ README covers architecture, agent roles, execution flow, technologies, local setup, env vars, API usage, deployment.

☐ A sample research query has been tested on the production URLs.

First action today: FastAPI project → Pydantic task models → PostgreSQL persistence → LangGraph state → Planner Agent → first working API request. Get one research query through the full backend workflow before polishing the UI.